

文档与视图

党宇

dangyu@nankai.edu.cn



机器人与信息自动化研究所

Institute of Robotics & Automatic Information System



南开大学

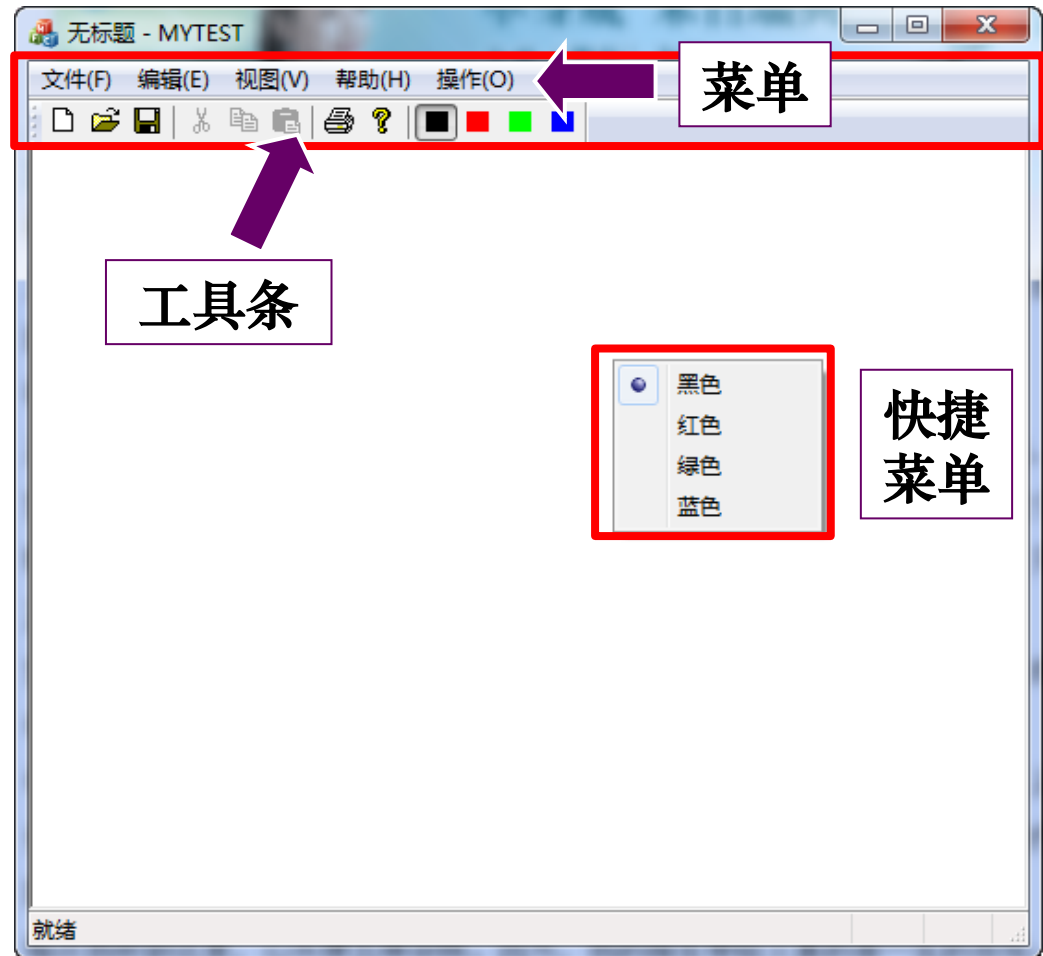
Nankai University

主要内容

- 文档与视图
- 资源的应用
- WM_TIMER消息
- 综合实例

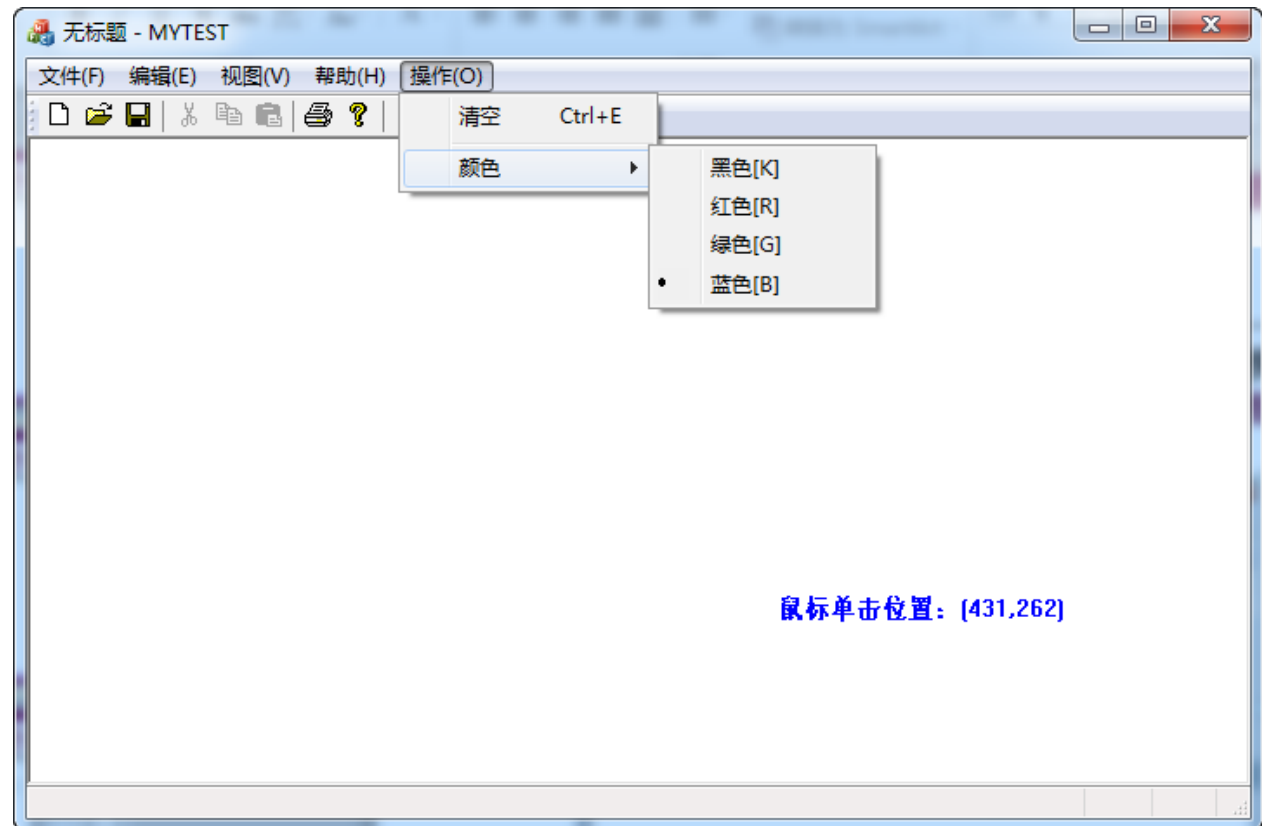
资源的应用

- 菜单资源
- 快捷菜单
- 快捷键资源
- 工具条资源



资源的应用

- 综合举例：继续鼠标单击例子，添加菜单项、快捷菜单、快捷键和工具条，实现**点击位置信息清空**、**改变字符串颜色**的功能



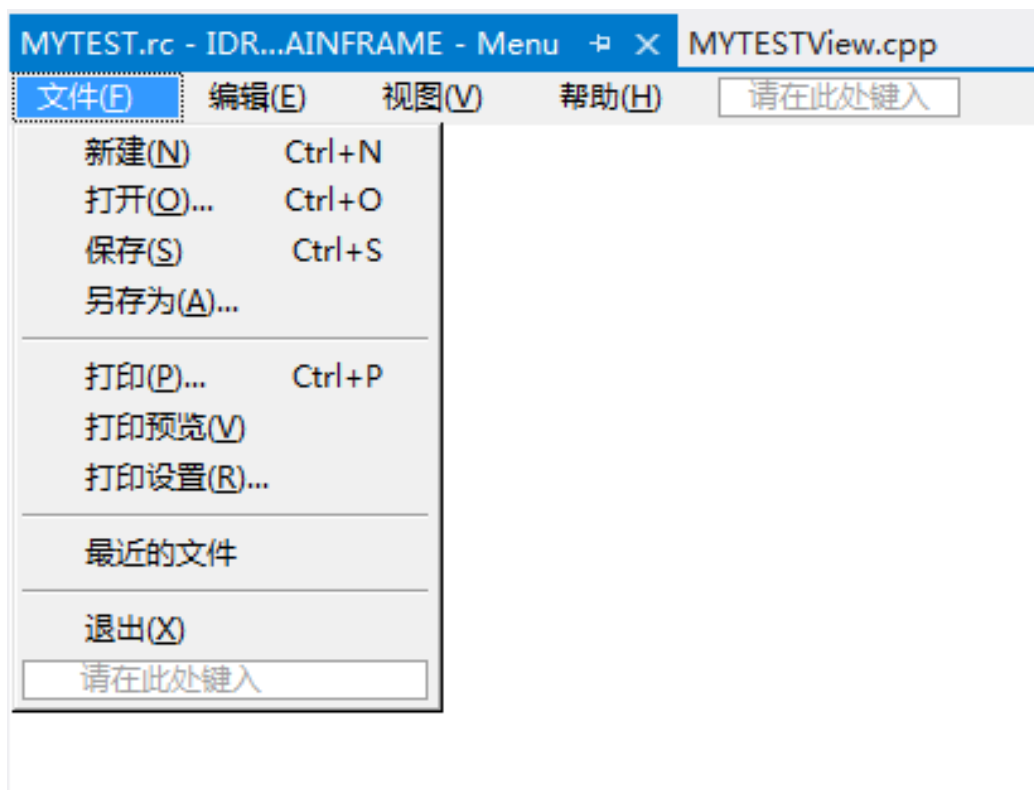
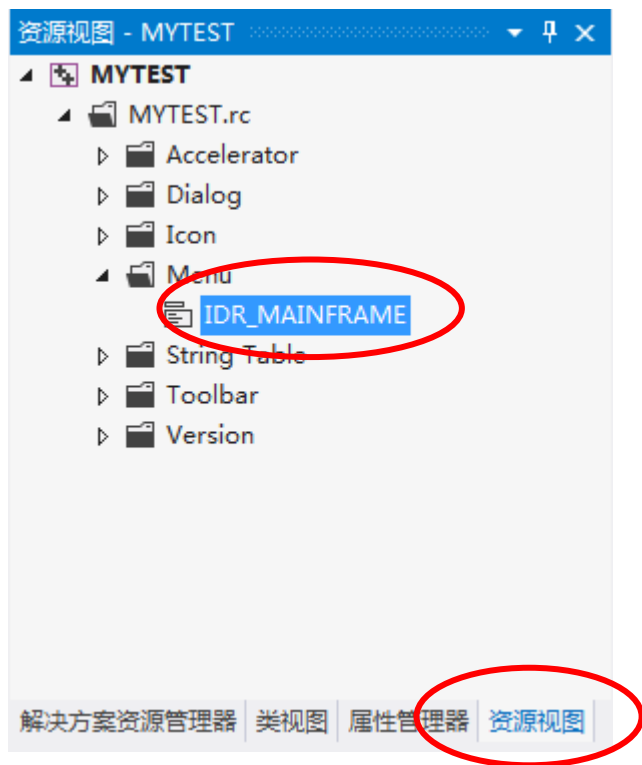
1、菜单资源

➤ 编程工作分析

- 1) 添加菜单资源，包括清空和颜色菜单项
- 2) 在Doc和View类中添加代表颜色的成员变量，并撰写相关函数
- 3) 响应菜单相关的消息，撰写消息响应函数

1、菜单资源

- 选择资源视图
- 选择Menu->IDR_MAINFRAME

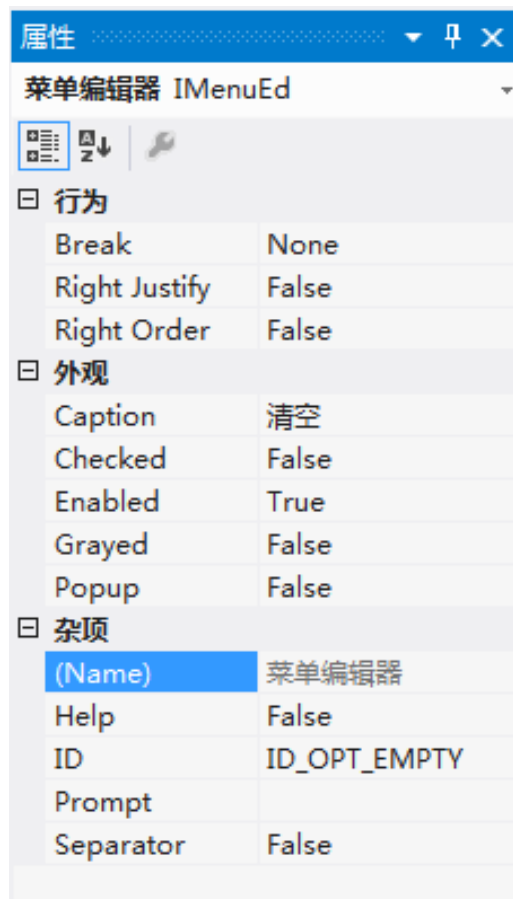


1、菜单资源

1. 添加菜单项

➤ 需要设置的按钮属性

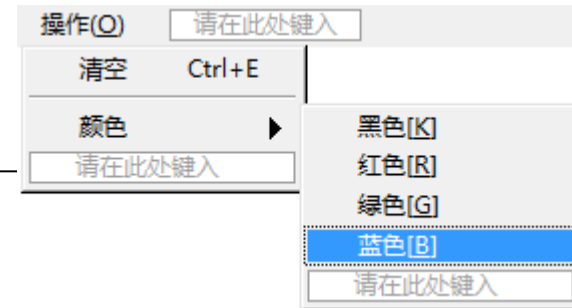
- Caption: 显示标题
- Checked: 表示菜单项是否被选中
- Grayed: 表示菜单项是否被禁止
- Popup: 菜单项是否为弹出式菜单
- ID: 标识, 根据实际意义修改ID
- Prompt: 提示内容
- Separator: 表示菜单项是分割线



属性	
菜单编辑器 IMenuEd	
行为	
Break	None
Right Justify	False
Right Order	False
外观	
Caption	清空
Checked	False
Enabled	True
Grayed	False
Popup	False
杂项	
(Name)	菜单编辑器
Help	False
ID	ID_OPT_EMPTY
Prompt	
Separator	False

1、菜单资源

1. 添加菜单项



ID	Caption	Separator	Pop-up
无	操作(&O)		yes
ID_OPT_EMPTY	清空 \tCtrl+E		
无	分割线	yes	
无	颜色		yes
ID_OPT_BLACK	黑色[&K]		
ID_OPT_RED	红色[&R]		
ID_OPT_GREEN	绿色[&G]		
ID_OPT_BLUE	蓝色[&B]		

1、菜单资源

1. 添加菜单项

确保四种颜色的ID对应的数值连续

Resource.h

// Microsoft Visual C++ 生成的包含文件。

// 供 MYTEST.rc 使用

//

#define ID_OPT_EMPTY 32776

#define ID_OPT_BLACK 32781

#define ID_OPT_RED 32782

#define ID_OPT_GREEN 32783

#define ID_OPT_BLUE 32784

系统添加

1、菜单资源

2. 添加成员变量和相应函数

在CMYTESTDoc类定义中(MYTESTDoc.h)添加

private:

```
int m_nColorIndex;           // 代表颜色
```

public:

```
void SetColor(int index);
```

```
int GetColor();
```

MYTESTDoc.cpp

```
CMYTESTDoc::CMYTESTDoc()
```

```
{
```

```
    m_str.Empty();
```

```
    m_point = CPoint(0,0);
```

```
    m_nColorIndex = 0;           // 默认黑色
```

```
}
```

添加蓝色
代码

1、菜单资源

2. 添加成员变量和相应函数

MYTESTDoc.cpp

```
void CMYTESTDoc::SetColor(int index)
{   m_nColorIndex = index; }
```

```
int CMYTESTDoc::GetColor()
{   return m_nColorIndex; }
```

```
void CMYTESTDoc::Serialize(CArchive& ar) {
    if(ar.IsStoring())           //写操作
        ar<<m_point<<m_str<<m_nColorIndex;
    else                          //读操作
        ar>>m_point>>m_str>>m_nColorIndex;
}
```

添加蓝色
代码

1、菜单资源

2. 添加成员变量和相应函数

在CMYTESTView类定义中(MYTESTView.h)添加

private:

```
COLORREF m_nColors[4];    // 颜色值
```

MYTESTView.cpp

```
CMYTESTView::CMYTESTView()
```

```
{
```

```
    m_nColors[0] = RGB(0,0,0);        // 黑色
```

```
    m_nColors[1] = RGB(255,0,0);      // 红色
```

```
    m_nColors[2] = RGB(0,255,0);      // 绿色
```

```
    m_nColors[3] = RGB(0,0,255);      // 蓝色
```

```
}
```

添加蓝色
代码

1、菜单资源

2. 添加成员变量和相应函数

MYTESTView.cpp

```
void CMYTESTView::OnDraw(CDC* pDC) {  
    CMYTESTDoc* pDoc = GetDocument();  
    ASSERT_VALID(pDoc);  
    if (!pDoc)  
        return;  
  
    pDC->SetTextColor(m_nColors[pDoc->GetColor()]);  
    CPoint pt;  
    CString str;  
    pDoc->Get(pt,str);  
    pDC->TextOut(pt.x,pt.y,str);  
}
```

添加蓝色
代码

1、菜单资源

3. 响应COMMAND消息

➤ 将菜单命令映射到哪个类?

- 将菜单命令映射到哪个类中，需要由该命令的功能决定
- 如果一个命令同视图的显示有关，就应该将其映射到视图类
- 如果同文档的读写有关，就映射到文档类中
- 如果命令完成通用功能，一般映射到窗口框架类
- 无法对功能进行准确分类时，则可以将菜单命令映射到任意一个类，看看是否能够完成指定的功能

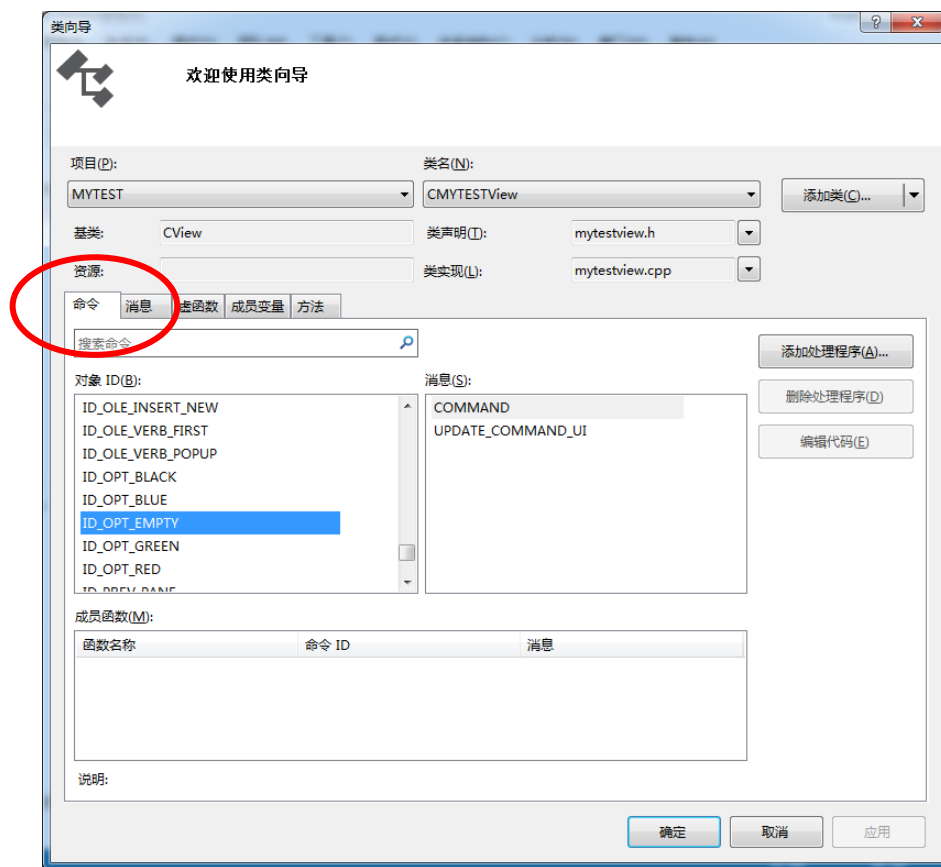
1、菜单资源

3. 响应COMMAND消息

用户单击菜单项时产生COMMAND消息

➤ 在Doc类中响应消息

- ID_OPT_EMPTY
- ID_OPT_BLACK
- ID_OPT_RED
- ID_OPT_GREEN
- ID_OPT_BLUE



1、菜单资源

3. 响应COMMAND消息

MYTESTDoc.cpp

```
void CMYTESTDoc::OnOptEmpty() // 清空
```

```
{  
    m_str.Empty();  
    UpdateAllViews(NULL);  
}
```

MYTESTDoc.cpp

```
void CMYTESTDoc::OnOptBlack()
```

```
{  
    m_nColorIndex = 0;  
    UpdateAllViews(NULL);  
}
```


1、菜单资源

4. 响应UPDATE_COMMAND_UI消息

窗口将要绘制菜单项时产生UPDATE_COMMAND_UI消息

- 在View类中响应UPDATE_COMMAND_UI消息
 - ID_OPT_BLACK
 - ID_OPT_RED
 - ID_OPT_GREEN
 - ID_OPT_BLUE
- 使用CCmdUI类设置菜单项状态

1、菜单资源

4. 响应UPDATE_COMMAND_UI消息

MYTESTView.cpp

```
void CMYTESTView::OnUpdateOptBlack(CCmdUI *pCmdUI){  
    CMYTESTDoc* pDoc=GetDocument();  
    pCmdUI->SetRadio(pDoc->GetColor() == 0);  
}
```

MYTESTView.cpp

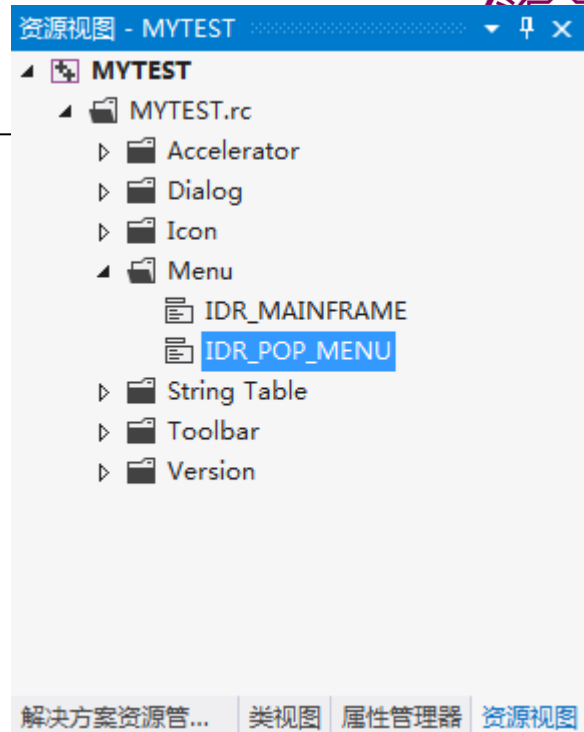
```
void CMYTESTView::OnUpdateOptRed(CCmdUI *pCmdUI){  
    CMYTESTDoc* pDoc=GetDocument();  
    pCmdUI->SetRadio(pDoc->GetColor() == 1);  
}
```

2、快捷菜单

➤ 方法一：在资源视图中新建菜单 IDR_POP_MENU

注意：快捷菜单ID值和对应菜单项的ID值相同

ID	Caption	Pop-up
无	POP	yes
ID_OPT_BLACK	黑色[&K]	
ID_OPT_RED	红色[&R]	
ID_OPT_GREEN	绿色[&G]	
ID_OPT_BLUE	蓝色[&B]	



2、快捷菜单

➤ 响应弹出菜单消息WM_CONTEXTMENU

- 消息中的坐标point是屏幕坐标

```
void CMYTESTView::OnContextMenu(CWnd* /*pWnd*/, CPoint point)
{
    CMYTESTDoc* pDoc=GetDocument();
    CMenu PopMenu;
    PopMenu.LoadMenu(IDR_POP_MENU);           // 装入菜单资源
    // 获得弹出式菜单指针
    CMenu *pPopupMenu=PopMenu.GetSubMenu(0);
    // 设置单选按钮
    pPopupMenu->CheckMenuItem(ID_OPT_BLACK, ID_OPT_BLUE,
    ID_OPT_BLACK+pDoc->GetColor(), MF_BYCOMMAND);
    // 显示菜单
    pPopupMenu->TrackPopupMenu(TPM_LEFTALIGN, point.x, point.y,
    this); }
```

2、快捷菜单

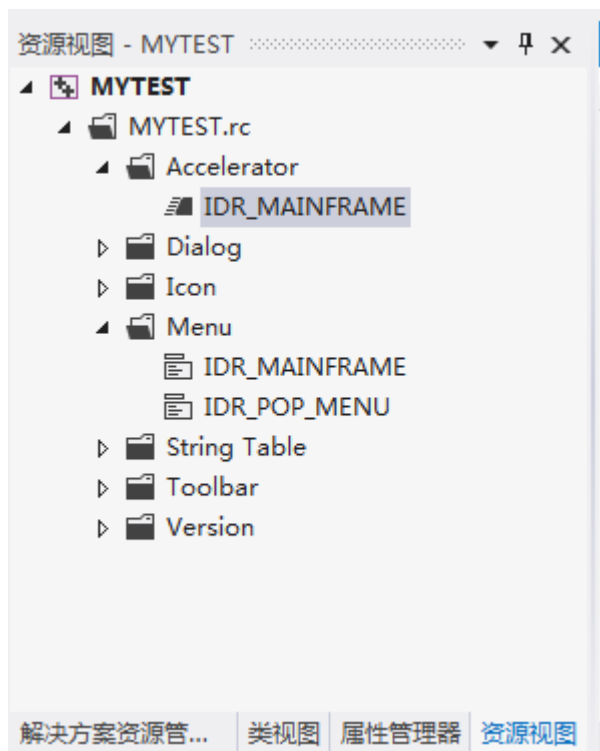
➤ 方法二：响应鼠标右键单击，在程序中新建菜单

```
void CMYTESTView::OnRButtonDown(UINT nFlags, CPoint point){  
    CMenu PopMenu;  
    PopMenu.CreatePopupMenu(); // 创建弹出式菜单，并附加到PopMenu  
    PopMenu.AppendMenu(MF_ENABLED, ID_OPT_BLACK, _T("黑色[&K]"));  
    PopMenu.AppendMenu(MF_DISABLED, ID_OPT_RED, _T("红色[&R]"));  
    PopMenu.AppendMenu(MF_DISABLED, ID_OPT_GREEN, _T("绿色[&G]"));  
    PopMenu.AppendMenu(MF_DISABLED, ID_OPT_BLUE, _T("蓝色[&B]"));  
  
    CMYTESTDoc* pDoc=GetDocument();  
    PopMenu.CheckMenuRadioItem(ID_OPT_BLACK, ID_OPT_BLUE,  
        ID_OPT_BLACK+pDoc->GetColor(), MF_BYCOMMAND);  
  
    ClientToScreen(&point); // 将客户区坐标转化为屏幕坐标  
    PopMenu.TrackPopupMenu(TPM_RIGHTALIGN, point.x, point.y, this);  
    CView::OnRButtonDown(nFlags, point); }  

```

3、快捷键资源

➤ 添加快捷键

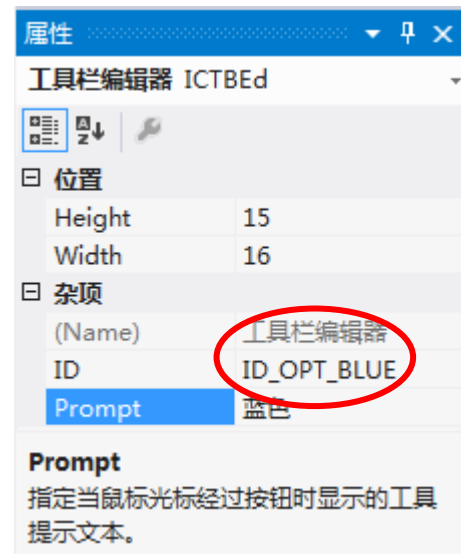
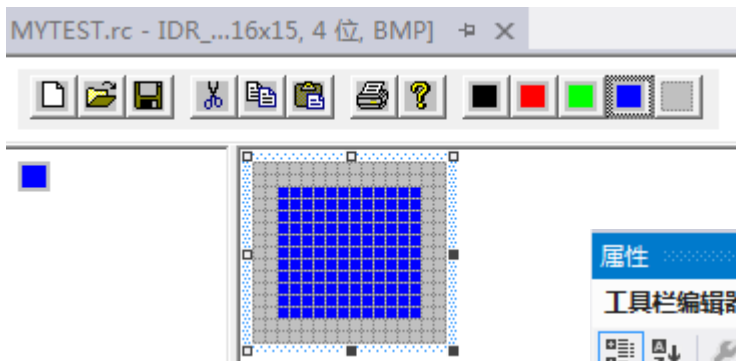
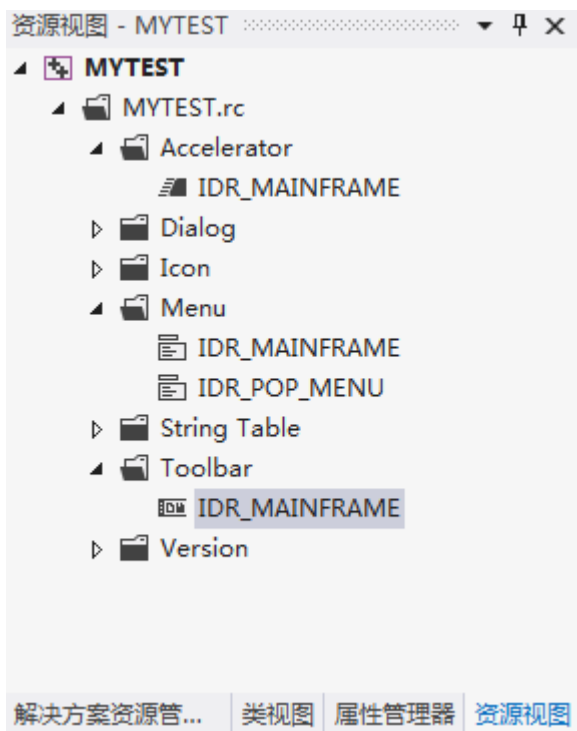


ID	修饰符	键	类型
ID_EDIT_COPY	Ctrl	C	VIRTKEY
ID_EDIT_COPY	Ctrl	VK_INSERT	VIRTKEY
ID_EDIT_CUT	Shift	VK_DELETE	VIRTKEY
ID_EDIT_CUT	Ctrl	X	VIRTKEY
ID_EDIT_PASTE	Ctrl	V	VIRTKEY
ID_EDIT_PASTE	Shift	VK_INSERT	VIRTKEY
ID_EDIT_UNDO	Alt	VK_BACK	VIRTKEY
ID_EDIT_UNDO	Ctrl	Z	VIRTKEY
ID_FILE_NEW	Ctrl	N	VIRTKEY
ID_FILE_OPEN	Ctrl	O	VIRTKEY
ID_FILE_PRINT	Ctrl	P	VIRTKEY
ID_FILE_SAVE	Ctrl	S	VIRTKEY
ID_NEXT_PANE	无	VK_F6	VIRTKEY
ID_PREV_PANE	Shift	VK_F6	VIRTKEY
ID_OPT_EMPTY	Ctrl	E	VIRTKEY

4、工具条资源

➤ 添加工具条

- 注意工具条ID值和对应菜单项的ID值相同



主要内容

- 文档与视图
- 资源的应用
- WM_TIMER消息
- 综合实例

WM_TIMER消息

- WM_TIMER消息是一种**计时器消息**，用于周期性处理
- 相关函数
 - SetTimer(): 创建或设置一个定时器，当预定的时间到达，系统将产生WM_TIMER消息，通过参数表明是哪个定时器

```
UINT_PTR SetTimer(  
    HWND hWnd,           // 窗口句柄  
    UINT_PTR nIDEvent,    // 定时器ID，设置多个定时器时，  
                          // 通过ID判断是哪个定时器  
    UINT nElapse,        // 定时器时间间隔,单位为毫秒  
    TIMERPROC lpTimerFunc // 回调函数  
);
```

WM_TIMER消息

- WM_TIMER消息是一种**计时器消息**，用于周期性处理
- 相关函数
 - KillTimer(): 销毁之前用SetTimer()设置的定时器，当定时器使用完毕后调用。

```
BOOL KillTimer(  
    HWND hwnd;           // 窗口句柄  
    UINT nIDEvent)       // 定时器ID
```

- 使用过程
 - 设置定时器: SetTimer(**1**,2000,NULL);
 - 使用定时器: 响应WM_TIMER消息，在OnTimer()中
if(nIDEvent==**1**)
 - 销毁定时器: KillTimer(**1**);

举例：使用WM_TIMER消息

- 在屏幕上显示数字，每秒数字加一
- 添加成员变量

C***View类定义中添加

```
private:  
    int m_num;
```

C***View类定义构造函数中添加

```
m_num = 0;
```

- 响应WM_CREATE消息

```
OnCreate()  
{  
    SetTimer(1,1000,NULL);  
}
```

举例：使用WM_TIMER消息

- 在屏幕上显示数字，每秒数字加一
- 响应WM_DESTROY消息

```
OnDestroy() {  
    KillTimer(1); }  
}
```

- 响应WM_TIMER消息

```
OnTimer(UINT_PTR nIDEvent) {  
    m_num++;  
    Invalidate(); }  
}
```

```
OnDraw(CDC* pDC) {  
    CString str;  
    str.Format(_T("计数: %d"),m_num);  
    pDC->TextOutW(100,100,str); }  
}
```

练习题

- 1.分析本节课的练习实例，练习独立写出代码。
- 2.阅读教科书第10章和第11章。